

Recursive Definitions and Structural Induction

Section 5.3

Section Summary

- Recursively Defined Functions
- Recursively Defined Sets and Structures
- Structural Induction
- Generalized Induction

Recursively Defined Functions

Definition: A recursive or inductive definition of a function consists of two steps.

- BASIS STEP: Specify the value of the function at zero.
- RECURSIVE STEP: Give a rule for finding its value at an integer from its values at smaller integers.
- A function $f(n)$ is the same as a sequence $a_0, a_1, \dots$, where a_i , where $f(i) = a_i$. This was done using recurrence relations in Section 2.4.

Recursively Defined Functions

Example: Suppose f is defined by:

$$f(0) = 3,$$

$$f(n+1) = 2f(n) + 3$$

Find $f(1), f(2), f(3), f(4)$

Solution:

- $f(1) = 2f(0) + 3 = 2 \cdot 3 + 3 = 9$
- $f(2) = 2f(1) + 3 = 2 \cdot 9 + 3 = 21$
- $f(3) = 2f(2) + 3 = 2 \cdot 21 + 3 = 45$
- $f(4) = 2f(3) + 3 = 2 \cdot 45 + 3 = 93$

Example: Give a recursive definition of the factorial function $n!$:

Solution:

$$f(0) = 1$$

$$f(n+1) = (n+1) \cdot f(n)$$

Recursively Defined Functions

Example: Give a recursive definition of:

$$\sum_{k=0}^n a_k.$$

Solution: The first part of the definition is

$$\sum_{k=0}^0 a_k = a_0.$$

The second part is

$$\sum_{k=0}^{n+1} a_k = \left(\sum_{k=0}^n a_k \right) + a_{n+1}.$$

Fibonacci Numbers

Fibonacci
(1170- 1250)



Example : The Fibonacci numbers are defined as follows:

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}$$

Find f_2, f_3, f_4, f_5 .

- $f_2 = f_1 + f_0 = 1 + 0 = 1$
- $f_3 = f_2 + f_1 = 1 + 1 = 2$
- $f_4 = f_3 + f_2 = 2 + 1 = 3$
- $f_5 = f_4 + f_3 = 3 + 2 = 5$

In Chapter 8, we will use the Fibonacci numbers to model population growth of rabbits. This was an application described by Fibonacci himself.

Next, we use strong induction to prove a result about the Fibonacci numbers.

Fibonacci Numbers

Example 4: Show that whenever $n \geq 3$, $f_n > \alpha^{n-2}$, where $\alpha = (1 + \sqrt{5})/2$.

Solution: Let $P(n)$ be the statement $f_n > \alpha^{n-2}$. Use strong induction to show that $P(n)$ is true whenever $n \geq 3$.

- **BASIS STEP:** $P(3)$ holds since $\alpha < 2 = f_3$
 $P(4)$ holds since $\alpha^2 = (3 + \sqrt{5})/2 < 3 = f_4$.
- **INDUCTIVE STEP:** Assume that $P(j)$ holds, i.e., $f_j > \alpha^{j-2}$ for all integers j with $3 \leq j \leq k$, where $k \geq 4$. Show that $P(k+1)$ holds, i.e., $f_{k+1} > \alpha^{k-1}$.

- Since $\alpha^2 = \alpha + 1$ (because α is a solution of $x^2 - x - 1 = 0$),

$$\alpha^{k-1} = \alpha^2 \cdot \alpha^{k-3} = (\alpha + 1) \cdot \alpha^{k-3} = \alpha \cdot \alpha^{k-3} + 1 \cdot \alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}$$

- By the inductive hypothesis, because $k \geq 4$ we have

$$f_{k-1} > \alpha^{k-3}, \quad f_k > \alpha^{k-2}.$$

- Therefore, it follows that

$$f_{k+1} = f_k + f_{k-1} > \alpha^{k-2} + \alpha^{k-3} = \alpha^{k-1}.$$

- Hence, $P(k+1)$ is true.

Why does this equality hold?

Euclidean Algorithm



Euclid

(325 B.C.E. – 265 B.C.E.)

- The Euclidian algorithm is an efficient method for computing the greatest common divisor of two integers. It is based on the idea that $\gcd(a, b)$ is equal to $\gcd(b, c)$ when $a > b$ and c is the remainder when a is divided by b .

Example: Find $\gcd(91, 287)$:

- $287 = 91 \cdot 3 + 14$ Divide 287 by 91
- $91 = 14 \cdot 6 + 7$ Divide 91 by 14
- $14 = 7 \cdot 2 + 0$ Divide 14 by 7

Stopping condition

$$\gcd(287, 91) = \gcd(91, 14) = \gcd(14, 7) = 7$$

continued →

Euclidean Algorithm

- The Euclidean algorithm expressed in pseudocode is:

```
procedure gcd( $a, b$ : positive integers)
```

```
   $x := a$ 
```

```
   $y := b$ 
```

```
  while  $y \neq 0$ 
```

```
     $r := x \bmod y$ 
```

```
     $x := y$ 
```

```
     $y := r$ 
```

```
  return  $x$  {gcd( $a, b$ ) is  $x$ }
```

- In Section 5.3, we'll see that the time complexity of the algorithm is $O(\log b)$, where $a > b$.

Gabriel Lamé
(1795-1870)



Lamé's Theorem

Lamé's Theorem: Let a and b be positive integers with $a \geq b$. Then the number of divisions used by the Euclidean algorithm to find $\gcd(a, b)$ is less than or equal to five times the number of decimal digits in b .

Proof: When we use the Euclidean algorithm to find $\gcd(a, b)$ with $a \geq b$,

- n divisions are used to obtain
(with $a = r_0, b = r_1$):

$$r_0 = r_1 q_1 + r_2 \quad 0 \leq r_2 < r_1,$$

$$r_1 = r_2 q_2 + r_3 \quad 0 \leq r_3 < r_2,$$

$$\vdots$$

$$r_{n-2} = r_{n-1} q_{n-1} + r_n \quad 0 \leq r_n < r_{n-1},$$

$$r_{n-1} = r_n q_n.$$

- Since each quotient $q_1, q_2, \dots, q_{n-1}$ is at least 1 and $q_n \geq 2$:

$$r_n \geq 1 = f_2,$$

$$r_{n-1} \geq 2 r_n \geq 2 f_2 = f_3,$$

$$r_{n-2} \geq r_{n-1} + r_n \geq f_3 + f_2 = f_4,$$

$$\vdots$$

$$r_2 \geq r_3 + r_4 \geq f_{n-1} + f_{n-2} = f_n,$$

$$b = r_1 \geq r_2 + r_3 \geq f_n + f_{n-1} = f_{n+1}.$$

continued $\rightarrow$

Lamé's Theorem

- It follows that if n divisions are used by the Euclidian algorithm to find $\gcd(a, b)$ with $a \geq b$, then $b \geq f_{n+1}$. By Example 4, $f_{n+1} > \alpha^{n-1}$, for $n > 2$, where $\alpha = (1 + \sqrt{5})/2$. Therefore, $b > \alpha^{n-1}$.
- Because $\log_{10} \alpha \approx 0.208 > 1/5$, $\log_{10} b > (n-1) \log_{10} \alpha > (n-1)/5$. Hence,

$$n-1 < 5 \cdot \log_{10} b.$$
- Suppose that b has k decimal digits. Then $b < 10^k$ and $\log_{10} b < k$. It follows that $n-1 < 5k$ and since k is an integer, $n \leq 5k$.
- As a consequence of Lamé's Theorem, $O(\log b)$ divisions are used by the Euclidian algorithm to find $\gcd(a, b)$ whenever $a > b$.
 - By Lamé's Theorem, the number of divisions needed to find $\gcd(a, b)$ with $a > b$ is less than or equal to $5(\log_{10} b + 1)$ since the number of decimal digits in b (which equals $\lfloor \log_{10} b \rfloor + 1$) is less than or equal to $\log_{10} b + 1$.

Lamé's Theorem was the first result in computational complexity

Recursively Defined Sets and Structures

Recursive definitions of sets have two parts:

- The *basis step* specifies an initial collection of elements.
- The *recursive step* gives the rules for forming new elements in the set from those already known to be in the set.
- Sometimes the recursive definition has an *exclusion rule*, which specifies that the set contains nothing other than those elements specified in the basis step and generated by applications of the rules in the recursive step.
- We will always assume that the exclusion rule holds, even if it is not explicitly mentioned.
- We will later develop a form of induction, called *structural induction*, to prove results about recursively defined sets.

Recursively Defined Sets and Structures

Example : Subset of Integers S :

BASIS STEP: $3 \in S$.

RECURSIVE STEP: If $x \in S$ and $y \in S$, then $x + y$ is in S .

- Initially 3 is in S , then $3 + 3 = 6$, then $3 + 6 = 9$, etc.

Example: The natural numbers $\mathbb{N}$.

BASIS STEP: $0 \in \mathbb{N}$.

RECURSIVE STEP: If n is in $\mathbb{N}$, then $n + 1$ is in $\mathbb{N}$.

- Initially 0 is in S , then $0 + 1 = 1$, then $1 + 1 = 2$, etc.

Strings

Definition: The set Σ^* of *strings* over the alphabet Σ :

BASIS STEP: $\lambda \in \Sigma^*$ (λ is the empty string)

RECURSIVE STEP: If w is in Σ^* and x is in Σ ,
then $wx \in \Sigma^*$.

Example: If $\Sigma = \{0,1\}$, the strings in Σ^* are the set of all bit strings, $\lambda, 0, 1, 00, 01, 10, 11$, etc.

Example: If $\Sigma = \{a,b\}$, show that aab is in Σ^* .

- Since $\lambda \in \Sigma^*$ and $a \in \Sigma$, $a \in \Sigma^*$.
- Since $a \in \Sigma^*$ and $a \in \Sigma$, $aa \in \Sigma^*$.
- Since $aa \in \Sigma^*$ and $b \in \Sigma$, $aab \in \Sigma^*$.

String Concatenation

Definition: Two strings can be combined via the operation of *concatenation*. Let Σ be a set of symbols and Σ^* be the set of strings formed from the symbols in Σ . We can define the concatenation of two strings, denoted by $\cdot$, recursively as follows.

BASIS STEP: If $w \in \Sigma^*$, then $w \cdot \lambda = w$.

RECURSIVE STEP: If $w_1 \in \Sigma^*$ and $w_2 \in \Sigma^*$ and $x \in \Sigma$, then $w_1 \cdot (w_2 x) = (w_1 \cdot w_2)x$.

- Often $w_1 \cdot w_2$ is written as $w_1 w_2$.
- If $w_1 = abra$ and $w_2 = cadabra$, the concatenation $w_1 w_2 = abracadabra$.

Length of a String

Example: Give a recursive definition of $l(w)$, the length of the string w .

Solution: The length of a string can be recursively defined by:

$$l(\lambda) = 0;$$

$$l(wx) = l(w) + 1 \text{ if } w \in \Sigma^* \text{ and } x \in \Sigma.$$

Balanced Parentheses

Example: Give a recursive definition of the set of balanced parentheses P .

Solution:

BASIS STEP: $() \in P$

RECURSIVE STEP: If $w \in P$, then $()w \in P$, $(w) \in P$ and $w() \in P$.

- Show that $((() ()))$ is in P .
- Why is $))((()$ not in P ?

Well-Formed Formulae in Propositional Logic

Definition: The set of *well-formed formulae* in propositional logic involving **T**, **F**, propositional variables, and operators from the set $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$.

BASIS STEP: **T**, **F**, and s , where s is a propositional variable, are well-formed formulae.

RECURSIVE STEP: If E and F are well formed formulae, then $(\neg E)$, $(E \wedge F)$, $(E \vee F)$, $(E \rightarrow F)$, $(E \leftrightarrow F)$, are well-formed formulae.

Examples: $((p \vee q) \rightarrow (q \wedge F))$ is a well-formed formula.
 $pq \wedge$ is not a well formed formula.

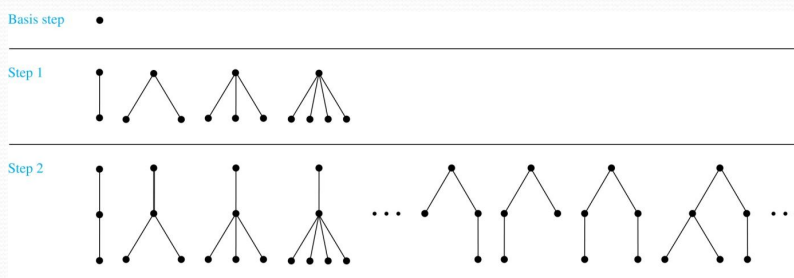
Rooted Trees (根树)

Definition: The set of *rooted trees*, where a rooted tree consists of a set of vertices containing a distinguished vertex called the *root*, and edges connecting these vertices, can be defined recursively by these steps:

BASIS STEP: A single vertex r is a rooted tree.

RECURSIVE STEP: Suppose that $T_1, T_2, \dots, T_n$ are disjoint rooted trees with roots $r_1, r_2, \dots, r_n$, respectively. Then the graph formed by starting with a root r , which is not in any of the rooted trees $T_1, T_2, \dots, T_n$, and adding an edge from r to each of the vertices $r_1, r_2, \dots, r_n$, is also a rooted tree.

Building Up Rooted Trees



- Trees are studied extensively in Chapter 11.
- Next we look at a special type of tree, the full binary tree.

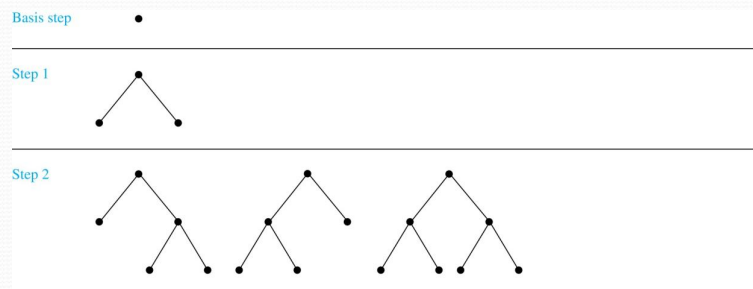
Full Binary Trees (满二叉树)

Definition: The set of *full binary trees* can be defined recursively by these steps.

BASIS STEP: There is a full binary tree consisting of only a single vertex r .

RECURSIVE STEP: If T_1 and T_2 are disjoint full binary trees, there is a full binary tree, denoted by $T_1 \cdot T_2$, consisting of a root r together with edges connecting the root to each of the roots of the left subtree T_1 and the right subtree T_2 .

Building Up Full Binary Trees



Induction and Recursively Defined Sets

Example: Show that the set S defined by specifying that $3 \in S$ and that if $x \in S$ and $y \in S$, then $x + y$ is in S , is the set of all positive integers that are multiples of 3.

Solution: Let A be the set of all positive integers divisible by 3. To prove that $A = S$, show that A is a subset of S and S is a subset of A .

- $A \subset S$: Let $P(n)$ be the statement that $3n$ belongs to S .
BASIS STEP: $3 \cdot 1 = 3 \in S$, by the first part of recursive definition.
INDUCTIVE STEP: Assume $P(k)$ is true. By the second part of the recursive definition, if $3k \in S$, then since $3 \in S$, $3k + 3 = 3(k + 1) \in S$. Hence, $P(k + 1)$ is true.
- $S \subset A$:
BASIS STEP: $3 \in S$ by the first part of recursive definition, and $3 = 3 \cdot 1$.
INDUCTIVE STEP: The second part of the recursive definition adds $x + y$ to S , if both x and y are in S . If x and y are both in A , then both x and y are divisible by 3. By part (i) of Theorem 1 of Section 4.1, it follows that $x + y$ is divisible by 3.
- We used mathematical induction to prove a result about a recursively defined set. Next we study a more direct form induction for proving results about recursively defined sets.

Structural Induction

Definition: To prove a property of the elements of a recursively defined set, we use *structural induction*.

BASIS STEP: Show that the result holds for all elements specified in the basis step of the recursive definition.

RECURSIVE STEP: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

- The validity of structural induction can be shown to follow from the principle of mathematical induction.

Full Binary Trees

Definition: The *height* $h(T)$ of a full binary tree T is defined recursively as follows:

- **BASIS STEP:** The height of a full binary tree T consisting of only a root r is $h(T) = 0$.
- **RECURSIVE STEP:** If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has height $h(T) = 1 + \max(h(T_1), h(T_2))$.
- The number of vertices $n(T)$ of a full binary tree T satisfies the following recursive formula:
 - **BASIS STEP:** The number of vertices of a full binary tree T consisting of only a root r is $n(T) = 1$.
 - **RECURSIVE STEP:** If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has the number of vertices $n(T) = 1 + n(T_1) + n(T_2)$.

Structural Induction and Binary Trees

Theorem: If T is a full binary tree, then $n(T) \leq 2^{h(T)+1} - 1$.

Proof: Use structural induction.

- **BASIS STEP:** The result holds for a full binary tree consisting only of a root, $n(T) = 1$ and $h(T) = 0$. Hence, $n(T) = 1 \leq 2^{0+1} - 1 = 1$.
- **RECURSIVE STEP:** Assume $n(T_1) \leq 2^{h(T_1)+1} - 1$ and also $n(T_2) \leq 2^{h(T_2)+1} - 1$ whenever T_1 and T_2 are full binary trees.

$$\begin{aligned}
 n(T) &= 1 + n(T_1) + n(T_2) && \text{(by recursive formula of } n(T)) \\
 &\leq 1 + (2^{h(T_1)+1} - 1) + (2^{h(T_2)+1} - 1) && \text{(by inductive hypothesis)} \\
 &\leq 2 \cdot \max(2^{h(T_1)+1}, 2^{h(T_2)+1}) - 1 \\
 &= 2 \cdot 2^{\max(h(T_1), h(T_2))+1} - 1 && (\max(2^x, 2^y) = 2^{\max(x, y)}) \\
 &= 2 \cdot 2^{h(T)+1} - 1 && \text{(by recursive definition of } h(T)) \\
 &= 2^{h(T)+1+1} - 1 \\
 &= 2^{h(T)+1} - 1
 \end{aligned}$$

Generalized (广义) Induction

- *Generalized induction* is used to prove results about sets other than the integers that have the well-ordering property. (explored in more detail in Chapter 9)
- For example, consider an ordering on $\mathbf{N} \times \mathbf{N}$, ordered pairs of nonnegative integers. Specify that (x_1, y_1) is less than or equal to (x_2, y_2) if either $x_1 < x_2$, or $x_1 = x_2$ and $y_1 < y_2$. This is called the *lexicographic ordering*.
- Strings are also commonly ordered by a *lexicographic ordering*.
- The next example uses generalized induction to prove a result about ordered pairs from $\mathbf{N} \times \mathbf{N}$.

Generalized Induction

Example: Suppose that $a_{m,n}$ is defined for $(m,n) \in \mathbf{N} \times \mathbf{N}$ by $a_{0,0} = 0$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1 & \text{if } n = 0 \text{ and } m > 0 \\ a_{m,n-1} + n & \text{if } n > 0 \end{cases}.$$

Show that $a_{m,n} = m + n(n+1)/2$ is defined for all $(m,n) \in \mathbf{N} \times \mathbf{N}$.

Solution: Use generalized induction.

BASIS STEP: $a_{0,0} = 0 = 0 + (0 \cdot 1)/2$

INDUCTIVE STEP: Assume that $a_{m',n'} = m' + n'(n'+1)/2$ whenever (m',n') is less than (m,n) in the lexicographic ordering of $\mathbf{N} \times \mathbf{N}$.

- If $n = 0$, by the inductive hypothesis we can conclude $a_{m,n} = a_{m-1,n} + 1 = m - 1 + n(n+1)/2 + 1 = m + n(n+1)/2$.
- If $n > 0$, by the inductive hypothesis we can conclude $a_{m,n} = a_{m,n-1} + n = m + n(n-1)/2 + n = m + n(n+1)/2$.

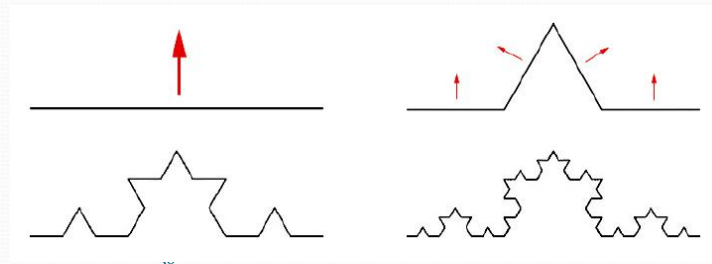
Fractals (分形)

- A fractal is a pattern that uses recursion
- The pattern itself repeats indefinitely



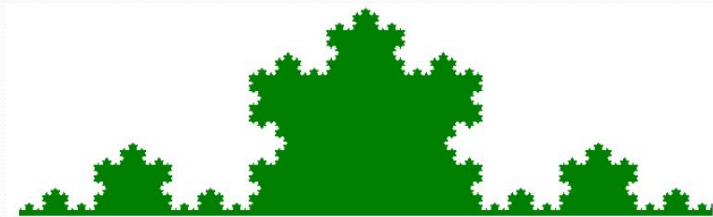
Fractals

- A geometric shape made up of same pattern repeated in different sizes and orientations
- Koch curve
 - A curve of order 0 is a straight line
 - A curve of order n consists of 4 curve of order $n-1$



Fractals

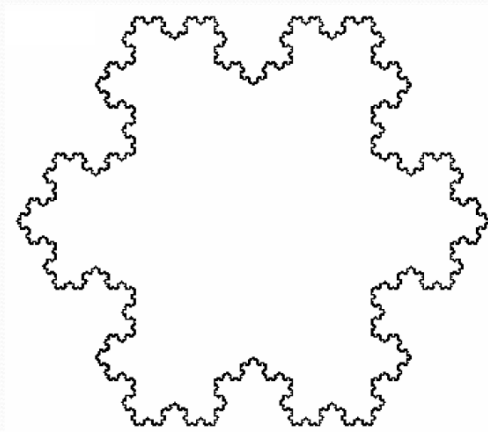
- Koch curve
 - after five iteration steps (order 4 curve)



31

Fractals

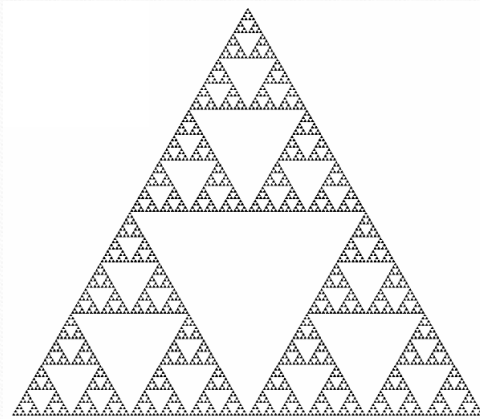
- Koch snowflake
 - From 3 Koch curves of order 4



32

Sierpinski Triangle

- A confined recursion of triangles to form a geometric lattice



33

Fibonacci Numbers and Nature

<http://www.maths.surrey.ac.uk/hosted-sites/R.Knott/Fibonacci/fibnat.html>

34

Homework

- 第8版: Sec. 5.3 6(a,d), 14, 31(a)